VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMACINIŲ SISTEMŲ INŽINERIJOS STUDIJŲ PROGRAMA

Optimizavimas be apribojimų

2 Laboratorinio darbo ataskaita

Atliko: Adomas Lukoševičius
VU el. p.: adomas.lukosevicius@mif.stud.vu.lt

Vertino: prof. dr. Julius Žilinskas

Vilnius
2026

UŽDUOTIS

1. SUPROGRAMUOKITE GRADIENTINIO NUSILEIDIMO, GREIČIAUSIOJO NUSILEIDIMO IR DEFORMUOJAMO SIMPLEKSO ALGORITMUS

Naudojant *Golang* programavimo kalbą, implementavau gradientinio nusileidimo, greičiausiojo nusileidimo ir deformuojamo simplekso algoritmus. Algoritmų kodai įkelti priede ataskaitos gale.

2. LAIKANT KINTAMAISIAIS DĖŽĖS PRIEKINĖS IR GALINĖS SIENŲ PLOTŲ SUMĄ, ŠONINIŲ SIENŲ PLOTŲ SUMĄ, VIRŠUTINĖS IR APATINĖS SIENŲ PLOTŲ SUMĄ, APRAŠYKITE VIENETINIO DĖŽĖS PAVIRŠIAUS PLOTO REIKALAVIMĄ IR DĖŽĖS TŪRIO PAKELTO KVADRATU FUNKCIJĄ.

Laikykime dėžės matmenis kintamaisiais x, y ir z. Pagal reikalavimus, galime nustatyti kintamuosius:

- $L = 2xz$ (priekinės ir galinės sienų plotų suma)
- $W = 2yz$ (šoninių sienų plotų suma)
- $H = 2xy$ (viršutinės ir apatinės sienų plotų suma)

Laikant visos dėžės plotą vienetiniu, akivaizdu, jog $L + W + H = 1$. Naudojantis dėžės tūrio formule ($V = L * W * H$) gauname tūrio, pakelto kvadratu funkciją:

$$V^2 = \frac{LWH}{8}$$

3. IŠ VIENETINIO PAVIRŠIAUS PLOTO REIKALAVIMO IŠVESKITE VIENO IŠ KINTAMOJO IŠRAIŠKĄ PER KITUS.

Pagal vienetinio paviršiaus ploto reikalavimą, gauname, kad $H = 1 – L – W$, o tai įsistatę į tūrio kvadratu funkciją gauname tikslo funkciją:

$$V^2 = \frac{LW(1 - L - W)}{8}$$

4. APRAŠYKITE TIKSLO FUNKCIJĄ *F*(*X*) TAIP, KAD OPTIMIZAVIMO UŽDAVINYS BŪTŲ FORMULUOJAMAS BE APRIBOJIMŲ: MIN *F*(*X*).

Kadangi anksčiau gauta tūrio pakelto kvadratu funkcija siekia maksimizuoti tūrį, t.y. ieškoma $max\ f(L, W)$, galime ją pakeisti minimizavimo funkcija, t.y. be apribojimų, ją paneigiant:

$$f(L, W) = -V^2 = -\frac{LW(1 - L - W)}{8}$$

Tikslo funkciją perrašiau *Golang* programavimo kalba (žr. pav. 1)

```go
func tiksloFunkcija(x, y float64) float64 {
    return -(x * y * (1 - x - y)) / 8
}
```

pav. 1 Tikslo funkcija

5. IŠVESKITE IR APRAŠYKITE TIKSLO FUNKCIJOS GRADIENTO FUNKCIJĄ.

Pagal anksčiau gautą tikslo funkciją $f(L, W) = -\frac{LW(1-L-W)}{8}$, apskaičiavau dalines išvestines:

- Nuo L: $\frac{\partial f}{\partial L} = \frac{W(2L+W-1)}{8}$
- Nuo W: $\frac{\partial f}{\partial W} = \frac{L(L+2W-1)}{8}$

Iš šių dalinių išvestinių gavau galutinę gradiento funkciją:

$$\nabla f(L,W) = \left(\frac{W(2L+W-1)}{8}, \frac{L(L+2W-1)}{8}\right)$$

Gradiento funkciją perrašiau *Golang* programavimo kalba (žr. pav. 2)

```go
func gradientoFunkcija(x, y float64) (dx, dy float64) {
    dx = x * (2*y + x - 1) / 8
    dy = y * (y + 2*x - 1) / 8
    return dx, dy
}
```

pav. 2 Gradiento funkcija

6. APSKAIČIUOKITE TIKSLO IR GRADIENTO FUNKCIJŲ REIKŠMES TAŠKUOSE X_0= (0,0), X_1=(1,1) IR X_M= (*A*/10, *B*/10), ČIA *A* IR *B* – STUDENTO KNYGELĖS NUMERIO "2X1XX*AB*" SKAITMENYS.

Pagal studento kodą, nustačiau A ir B reikšmes 3 ir 9 (žr. pav. 3).

```go
// mano studento skaitmenys
var A float64 = 3
var B float64 = 9
```

pav. 3 A ir B reikšmės

Naudojant *Golang,* apskaičiavau tikslo ir gradiento funkcijų rreikšmes taškuose X_0= (0,0), X_1=(1,1) ir X_m= (3/10, 9/10) (žr. pav. 4)

```
**taskas (0, 0)**
Tikslo funkcija: -0
Gradiento funkcija:(-0, -0)

**taskas (1, 1)**
Tikslo funkcija: 0.125
Gradiento funkcija:(0.25, 0.25)

**taskas (A/10, B/10)**
Tikslo funkcija: 0.00675
Gradiento funkcija:(0.04125, 0.05625)
```

pav. 4 Tikslo ir gradiento funkcijų reikšmės taškuose X_0, X_1 ir X_m

7. MINIMIZUOKITE SUFORMULUOTĄ UŽDAVINĮ NAUDOJANT SUPROGRAMUOTUS OPTIMIZAVIMO ALGORITMUS PRADEDANT IŠ TAŠKŲ X_0, X_1 IR X_M.

Naudojant ansčiau parašytus gradientinio nusileidimo, greičiausiojo nusileidimo ir deformuojamo simplekso algoritmus, minimizavau tikslo funkciją, pradedant taškuose X_0= (0,0), X_1=(1,1) ir X_m= (3/10, 9/10) ir atlikau algoritmų efektyvumo tyrimą (žr. pav. 5)

```
***Gradientinio nusileidimo algoritmas***
X0 X=(0.000000,0.000000) dev=4.630e-03 iter=0 fn=1 grad=1
X1 X=(0.333339,0.333339) dev=3.811e-12 iter=391 fn=392 grad=392
Xm X=(0.333316,0.333350) dev=1.184e-11 iter=1212 fn=1213 grad=1213

***Greiciausiojo nusileidimo algoritmas***
X0 X=(0.000000,0.000000) dev=4.630e-03 iter=0 fn=1 grad=1
X1 X=(0.333333,0.333333) dev=6.939e-18 iter=1 fn=51 grad=2
Xm X=(0.333330,0.333343) dev=2.891e-12 iter=16 fn=831 grad=17

***Deformuojamo simplekso algoritmas***
X0 X=(0.333333,0.333333) dev=3.867e-15 iter=50 fn=96 grad=0
X1 X=(0.333334,0.333333) dev=1.221e-15 iter=252 fn=474 grad=0
Xm X=(0.333333,0.333334) dev=4.605e-15 iter=201 fn=373 grad=0
```

pav. 5 Tyrimo terminalo išvestis

8. PALYGINKITE REZULTATUS: GAUTI SPRENDINIAI, RASTAS FUNKCIJOS MINIMUMO ĮVERTIS, ATLIKTŲ ŽINGSNIŲ IR FUNKCIJŲ SKAIČIAVIMŲ SKAIČIUS PRIKLAUSOMAI NUO PRADINIO TAŠKO.

Atlikus tyrimą, palyginau gautus rezultatus. Svarbu paminėti, kad tikrasis funkcijos minimumo taškas X^*, su kuriuo bus lyginamas algoritmų efektyvumas, buvo $(\frac{1}{3}, \frac{1}{3})$.

8.1. Gradientinio nusileidimo algoritmas

- Geriausiu atveju, algoritmas atliko 391 iteraciją, 392 kartus apskaičiavo $f(X)$ ir 392 kartus apskaičiavo $\nabla f(X)$.
- Gavo galutinę vertę 3,811 x 10^{-12} paklaida nuo tikrosios

- Lyginant su kitais:
    - Atliko daugiausia skaičiavimų iš visų algoritmų
    - Mažiausias tikslumas iš visų algoritmų

8.2. Greiciausiojo nusileidimo algoritmas

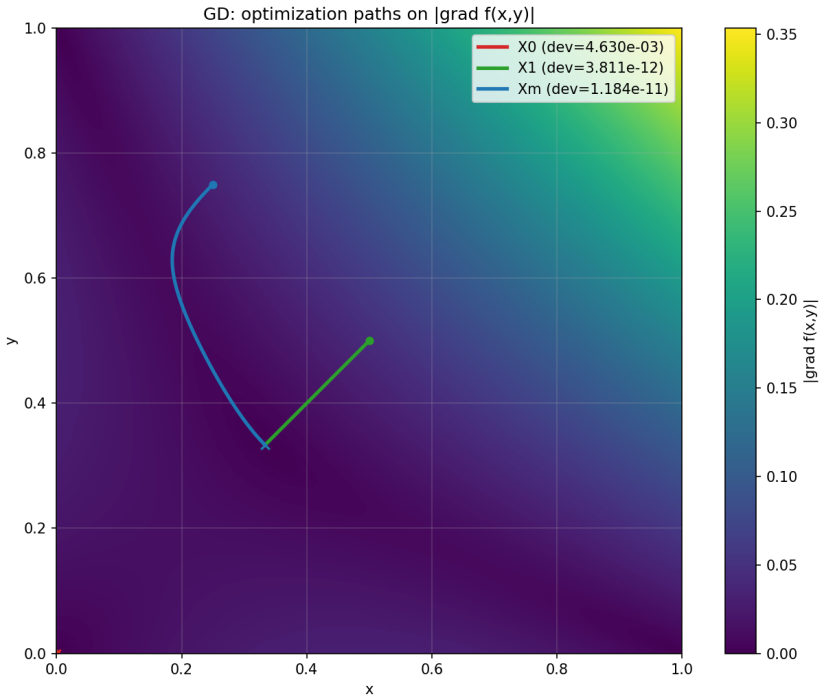
- Geriausiu atveju, algoritmas atliko 1 iteraciją, 52 kartus apskaičiavo *f(X)* ir 2 kartus apskaičiavo *∇f(X)*.
- Gavo galutinę vertę 6,929 x 10^{-18} paklaida nuo tikrosios
- Lyginant su kitais:
    - Atliko daug mažiau skaičiavimų negu kiti algoritmai – tik 1 iteracija, 2 gradiento funkcijos skaičiavimai
    - Gavo ženkliai tikslesnę galutinę vertę už kitus algoritmus
    - Efektyviausias iš testuotų algoritmų

8.3. Deformuojamo simplekso algoritmas

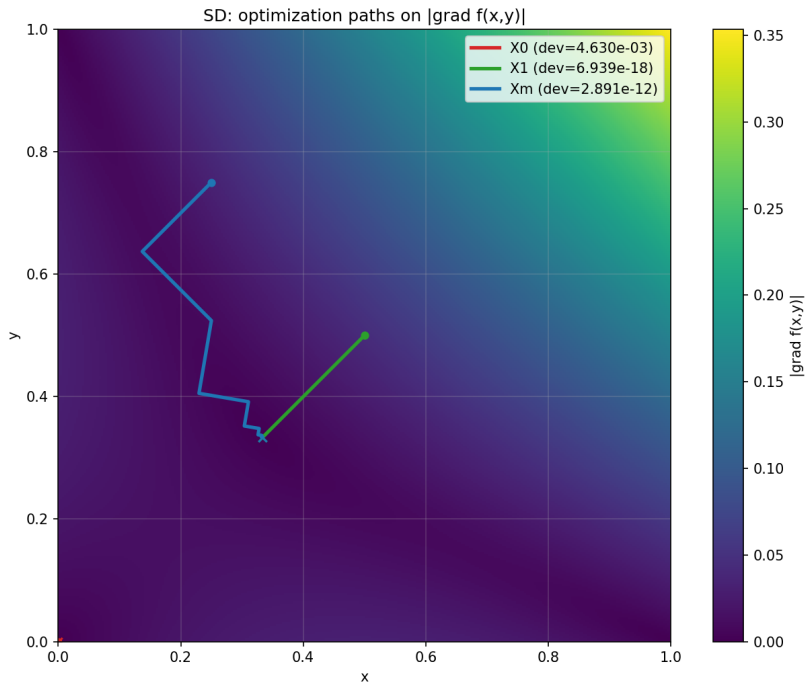
- Geriausiu atveju, algoritmas atliko 252 iteracijas, 474 kartus apskaičiavo *f(X)* ir 0 kartų apskaičiavo *∇f(X)*.
- Gavo galutinę vertę 1,221 x 10^{-15} paklaida nuo tikrosios
- Lyginant su kitais:
    - Viduryje tarp ankstesnių algoritmų pagal skaičiavimus ir tikslumą
    - Visiškai nenaudojo gradiento funkcijos

9. VIZUALIZUOKITE TIKSLO FUNKCIJĄ IR BANDYMO TAŠKUS.

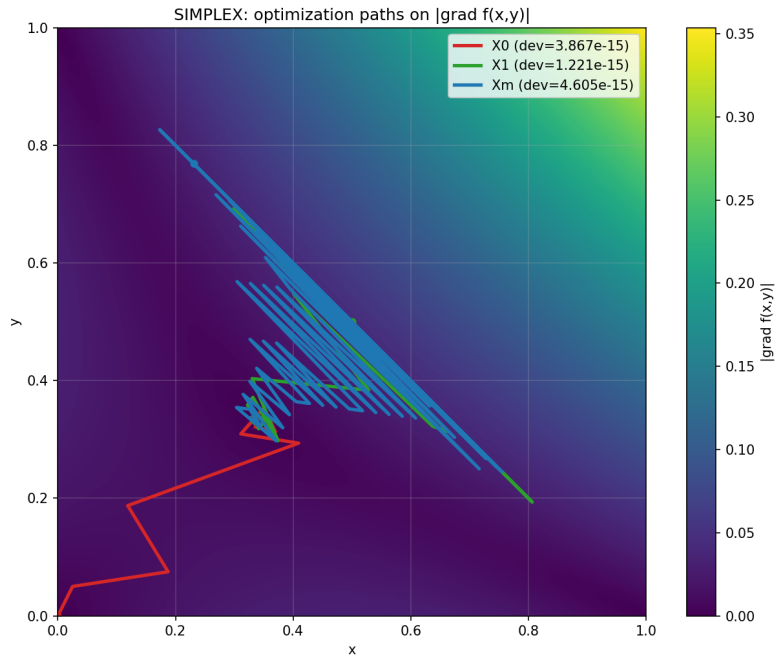
Atlikus tyrimą, gautus duomenis eksportavau į tekstinį failą ir naudojantis *Python pyplot* biblioteką kiekvienam algoritmui nubraižiau grafiką, parodant gradiento funkcijos rezultatą grafiko fone. Sugeneruoti grafikai pateikti žemiau.



pav. 6 Gradientinio nusileidimo algoritmo rezultatai



pav. 7 Greičiausiojo nusileidimo algoritmo rezultatai



pav. 8 Deformuojamo simplekso algoritmo rezultatai

PRIEDAS

1. GRADIENTINIO NUSILEIDIMO ALGORITMO KODAS

```
func gradientinioNusileidimoAlgo(f funkcija, gf gradientas, p0 point, gamma, eps
float64, maxIter int) (p point, value float64, k int, history []point) {
    p = project(p0)
```

```
    value = f(p)
    history = append(history, p)

    for k = 0; k < maxIter; k++ {
        g := gf(p)
        if math.Hypot(g.x, g.y) <= eps {
            break
        }

        p = project(point{x: p.x - gamma*g.x, y: p.y - gamma*g.y})

        value = f(p)
        history = append(history, p)
    }

    return p, value, k, history
}
```

2. GREIČIAUSIOJO NUSILEIDIMO ALGORITMO KODAS

```
func greiciausiojoNusileidimoAlgo(f funkcija, gf gradientas, p0 point, eps,
epsGamma, initialStep, maxGamma float64, maxIter int) (p point, value float64, k
int, history []point) {
    p = project(p0)

    value = f(p)
    history = append(history, p)

    for k = 0; k < maxIter; k++ {
        g := gf(p)
        if math.Hypot(g.x, g.y) <= eps {
            break
        }

        s := point{x: -g.x, y: -g.y}
        phi := func(gamma float64) float64 {
            cand := project(point{x: p.x + gamma*s.x, y: p.y + gamma*s.y})
            return f(cand)
        }

        a := 0.0
        b := initialStep
        fa := phi(a)
        fb := phi(b)

        for b < maxGamma && fb < fa {
            a = b
            fa = fb
            b *= 2
            if b > maxGamma {
```

```
                b = maxGamma
            }
            fb = phi(b)
        }

        if a == b {
            b = maxGamma
        }

        tau := (math.Sqrt(5) - 1) / 2
        c := b - tau*(b-a)
        d := a + tau*(b-a)
        fc := phi(c)
        fd := phi(d)

        for i := 0; i < 200 && (b-a) > epsGamma; i++ {
            if fc < fd {
                b = d
                d = c
                fd = fc
                c = b - tau*(b-a)
                fc = phi(c)
            } else {
                a = c
                c = d
                fc = fd
                d = a + tau*(b-a)
                fd = phi(d)
            }
        }

        gamma := (a + b) / 2
        p = project(point{x: p.x + gamma*s.x, y: p.y + gamma*s.y})

        value = f(p)
        history = append(history, p)
    }

    return p, value, k, history
}
```

3. DEFORMUOJAMO SIMPLEKSO ALGORITMO KODAS

```
func deformuojamoSimpleksoAlgo(f funkcija, p0 point, alpha, gamma, beta, eps
float64, maxIter int) (best point, bestValue float64, k int, history []point) {
    x1 := project(p0)
    x2 := project(point{x: p0.x + alpha, y: p0.y})
    x3 := project(point{x: p0.x, y: p0.y + alpha})

    f1 := f(x1)
    f2 := f(x2)
```

```go
    f3 := f(x3)

    // pradinis "best" ir istoriją
    best = x1
    bestValue = f1
    if f2 < bestValue {
        best = x2
        bestValue = f2
    }
    if f3 < bestValue {
        best = x3
        bestValue = f3
    }
    history = append(history, best)

    for k = 0; k < maxIter && simplexSize(x1, x2, x3) > eps; k++ {
        vertices := [3]point{x1, x2, x3}
        values := [3]float64{f1, f2, f3}

        for i := 0; i < 2; i++ {
            for j := i + 1; j < 3; j++ {
                if values[j] < values[i] {
                    values[i], values[j] = values[j], values[i]
                    vertices[i], vertices[j] = vertices[j], vertices[i]
                }
            }
        }

        xBest := vertices[0]
        xMid := vertices[1]
        xWorst := vertices[2]
        fBest := values[0]
        fMid := values[1]
        fWorst := values[2]

        xc := point{x: (xBest.x + xMid.x) / 2, y: (xBest.y + xMid.y) / 2}

        xr := project(point{x: 2*xc.x - xWorst.x, y: 2*xc.y - xWorst.y})
        fr := f(xr)

        if fr < fBest {
            xe := project(point{x: xc.x + gamma*(xr.x-xc.x), y: xc.y +
gamma*(xr.y-xc.y)})
            fe := f(xe)

            if fe < fr {
                xWorst = xe
                fWorst = fe
            } else {
                xWorst = xr
                fWorst = fr
```

```
            }
        } else if fBest <= fr && fr < fMid {
            xWorst = xr
            fWorst = fr
        } else {
            xCand := project(point{x: xc.x + beta*(xWorst.x-xc.x), y: xc.y +
beta*(xWorst.y-xc.y)})
            fCand := f(xCand)

            if fCand < fWorst {
                xWorst = xCand
                fWorst = fCand
            } else {
                xMid = point{x: xBest.x + 0.5*(xMid.x-xBest.x), y: xBest.y +
0.5*(xMid.y-xBest.y)}
                xWorst = point{x: xBest.x + 0.5*(xWorst.x-xBest.x), y: xBest.y +
0.5*(xWorst.y-xBest.y)}
                xMid = project(xMid)
                xWorst = project(xWorst)

                fMid = f(xMid)
                fWorst = f(xWorst)
            }
        }

        x1, x2, x3 = xBest, xMid, xWorst
        f1, f2, f3 = fBest, fMid, fWorst
        history = append(history, x1)
    }

    best = x1
    bestValue = f1
    if f2 < bestValue {
        best = x2
        bestValue = f2
    }
    if f3 < bestValue {
        best = x3
        bestValue = f3
    }

    return best, bestValue, k, history
}
```